

Lab 9

Buffer Overflow

Aim:

To access unprivileged memory

Theory:

Buffer overflow vulnerabilities in C programs often arise from the unsafe use of functions like `gets()`, which reads input from the user without any bounds checking. When a program uses `gets()` to store user input in a fixed-size buffer, an attacker can supply more data than the buffer can hold, overwriting adjacent memory locations. This can corrupt variables, alter the program's execution flow, or even inject malicious code if the overflow reaches the return address on the stack. The lack of length validation in `gets()` makes it particularly dangerous, as it will continue writing input into memory until it encounters a newline or EOF, regardless of the buffer's capacity.

The consequences of such buffer overflows can be severe, allowing attackers to execute arbitrary code or crash the program. Modern systems implement protections like stack canaries and address space layout randomization (ASLR) to mitigate these risks, but the fundamental issue remains when programmers use unsafe functions. Secure alternatives like `fgets()` should always be preferred, as they allow specifying a maximum number of characters to read, preventing buffer overflows. Understanding these vulnerabilities is crucial for writing robust C code and recognizing the importance of proper input validation and memory management.

Lab 1

Implementation of Nmap

Aim:

To explore the features provided by the network discovery tool NMAP

Theory:

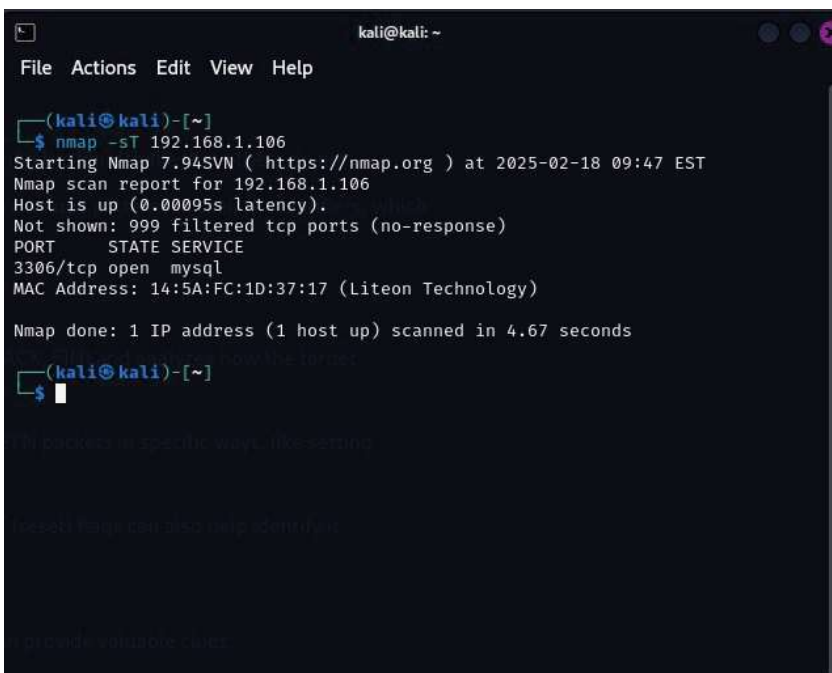
NMAP short for Network Mapper, is a FOSS (free and Open Source Software) network discovery and security auditing tool. It was released in september 1997 and is still used till today. It's grown to be an industry standard tool and is key in a cybersecurity professionals tool-belt. Nmap scans are considered "noisy" and are almost always detected by modern Intrusion detection systems (IDS). The purpose of an nmap scan is to find as many vulnerabilities as quickly as possible.

Procedure:

1. Start the kali linux virtual machine
2. Open a new Terminal windows
3. Run an NMAP scan on the local host using the Nmap command

Output

TCP scan



```
kali@kali: ~  
File Actions Edit View Help  
(kali@kali)-[~]  
$ nmap -sT 192.168.1.106  
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-02-18 09:47 EST  
Nmap scan report for 192.168.1.106  
Host is up (0.00095s latency).  
Not shown: 999 filtered tcp ports (no-response)  
PORT      STATE SERVICE  
3306/tcp  open  mysql  
MAC Address: 14:5A:FC:1D:37:17 (Liteon Technology)  
  
Nmap done: 1 IP address (1 host up) scanned in 4.67 seconds  
(kali@kali)-[~]  
$
```

UDP scan

```
kali@kali: ~  
File Actions Edit View Help  
  
(kali@kali)-[~]  
$ nmap -sU 192.168.1.106  
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-02-18 09:49 EST  
Nmap scan report for 192.168.1.106  
Host is up (0.00087s latency).  
Not shown: 999 open|filtered udp ports (no-response)  
PORT      STATE SERVICE  
5353/udp  open  zeroconf  
MAC Address: 14:5A:FC:1D:37:17 (Liteon Technology)  
  
Nmap done: 1 IP address (1 host up) scanned in 14.66 seconds  
  
(kali@kali)-[~]  
$
```

Lab 2

Hash cracking using John The ripper

Aim:

To learn how to crack encrypted zip files in multiple ways using John the Ripper

Theory:

John the ripper (JTR) is a free and open source software. It's intended to detect and crack weak unix passwords. It is available for many flavours of Unix, macOS, Windows, DOS and some more. A dictionary attack is a common method used in password cracking. As most people set simple passwords. JTR guesses words or phrases from a predefined list (called a word list or a dictionary) This type of attack relies on the assumption that the user may have chosen a common, weak or duplicated password. The hacker prepares a list of possible passwords such as common passwords, words from a dictionary, leaked passwords from previous data breaches or variations some variations to each of those. This can be done either manually or through a pre-existing list. They can then deploy the attack.

Procedure:

1. Create a plain text file and a word list of possible passwords file using the cat command
2. Zip it using the zip command and the --password flag to encrypt it
3. Using zip2john convert the encrypted file into a hash
4. Using the John command, load the password file and initiate the attack onto the hash file
5. Use John --show to see if your attack was successful

Output:

```
(tw1ster@EHPractice)-[~/Documents/Semester 8/Vuln/Lab]
$ cat > plain.txt
my passwords are:
password 123
chickenNugget
^C

(tw1ster@EHPractice)-[~/Documents/Semester 8/Vuln/Lab]
$ touch plain.txt

(tw1ster@EHPractice)-[~/Documents/Semester 8/Vuln/Lab]
$ cat plain.txt
my passwords are:
password 123
chickenNugget

(tw1ster@EHPractice)-[~/Documents/Semester 8/Vuln/Lab]
$ zip --password 777703 encrypted.zip plain.txt
adding: plain.txt (deflated 11%)

(tw1ster@EHPractice)-[~/Documents/Semester 8/Vuln/Lab]
$ ls
Comprehensive.odt  encrypted.zip  plain.txt  possiblePassword.txt

(tw1ster@EHPractice)-[~/Documents/Semester 8/Vuln/Lab]
$ 
(tw1ster@EHPractice)-[~/Documents/Semester 8/Vuln/Lab]
$ zip2john encrypted.zip > hash.txt
ver 2.0 efh 5455 efh 7875 encrypted.zip/plain.txt PKZIP Encr: TS_chk, cmplen=52, decmplen=45, crc=0AE9C5C7 ts=AD25 cs=ad25 type=8

(tw1ster@EHPractice)-[~/Documents/Semester 8/Vuln/Lab]
$ ls
Comprehensive.odt  encrypted.zip  hash.txt  plain.txt  possiblePassword.txt
```

```
(tw1ster@EHPractice)-[~/Documents/Semester 8/Vuln/Lab]
$ john --wordlist=possiblePassword.txt hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (PKZIP [32/64])
Will run 16 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
Warning: Only 6 candidates left, minimum 16 needed for performance.
777703 (encrypted.zip/plain.txt)
1g 0:00:00:00 DONE (2025-04-01 14:45) 100.0g/s 600.0p/s 600.0c/s 600.0C/s chickenNugget..knight
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

```
(tw1ster@EHPractice)-[~/Documents/Semester 8/Vuln/Lab]
$ john --show hash.txt
encrypted.zip/plain.txt:777703:plain.txt:encrypted.zip::encrypted.zip
1 password hash cracked, 0 left
```

Lab 3

Implementation of Nessus

Aim:

To scan a vulnerable system using nessus

Theory:

Nessus provides a powerful platform for identifying security weaknesses across networks and systems. As an industry-leading vulnerability scanner, Nessus automates the detection of missing patches, misconfigurations, and known vulnerabilities by comparing system characteristics against its extensive plugin database. The tool supports both credentialed scans (for deep system inspection) and unauthenticated scans (for surface-level assessment), generating comprehensive reports that categorize risks by severity. This process enables organizations to maintain visibility of their attack surface, comply with security standards, and prioritize remediation efforts effectively.

Limitations:

- Cannot detect zero-day vulnerabilities
- May produce false positives/negatives
- Scanning may impact network performance
- Requires regular plugin updates for optimal detection

Procedure:

1. Create a new Nessus account and log in on your local machine
2. Navigate to the Scans tab and create a new scan
3. Select “Basic Network Scan” and set the targets to be the local IP address followed by “/24”
4. Run the scan

Output:

FOLDERS

My Scans

All Scans

Trash

RESOURCES

Policies

Plugin Rules

Terrascan

Lab 4 / 192.168.1.107

Configure

Vulnerabilities 20

Filter Search Vulnerabilities 20 Vulnerabilities

Sev	CVSS	VPR	EPSS	Name	Family	Count
INFO	5.3			SMB Sig...	Misc.	1
INFO	...	...	...	SM...	Windows	6
INFO	...	...	...	SSL...	General	6
INFO	...	...	...	TLS...	General	3
INFO	...	...	...	HTT...	Web Servers	2
INFO	...	...	...	Mic...	Windows	2
INFO	...	...	...	TLS...	Service detection	2
INFO				Netstat ...	Port scanners	49
INFO				DCE Ser...	Windows	8
INFO				Service ...	Service detection	5
INFO				Device T...	General	1
INFO				Host Full...	General	1
INFO				mDNS D...	Service detection	1
INFO				MySQL S...	Databases	1
INFO				Nessus S...	Service detection	1
INFO				Netstat ...	General	1
INFO				OS Finge...	General	1
INFO				OS Ident...	General	1
INFO				OS Ident...	Misc.	1
INFO				Strict Tra...	Service detection	1

Host: 192.168.1.107

Host Details

IP: 192.168.1.107

OS: Windows 11

Start: Today at 8:21 PM

Vulnerabilities

Critical

High

Medium

Low

Info

tenable

Nessus Essentials

Scans

Settings

scott

FOLDERS

My Scans

All Scans

Trash

RESOURCES

Policies

Plugin Rules

Terrascan

Lab 4 / Plugin #11002

Configure

Vulnerabilities 17

INFO DNS Server Detection

Description

The remote service is a Domain Name System (DNS) server, which provides a mapping between hostnames and IP addresses.

Solution

Disable this service if it is not needed or restrict access to internal hosts only if the service is available externally.

See Also

https://en.wikipedia.org/wiki/Domain_Name_System

Output

No output recorded.

To see debug logs, please visit individual host

Port	Hosts
53 / tcp / dns	192.168.1.1
53 / udp / dns	192.168.1.1

Plugin Details

Severity: Info

ID: 11002

Version: \$Revision: 1.22 \$

Type: remote

Family: DNS

Published: February 13, 2003

Modified: May 16, 2017

Risk Information

Risk Factor: None

Lab 4

Using Burp suite to Intercept Web Requests

Aim:

To intercept and modify a request between the browser and a server using Burp Suite

Theory:

Intercepting and modifying web traffic with Burp Suite demonstrates the fundamental vulnerability of unsecured data transmission. As one of the most popular web proxy tools, Burp Suite allows security professionals to intercept HTTP/S requests between a browser and web server, revealing how easily transmitted data can be manipulated. The proxy functionality captures all traffic, including form submissions, API calls, and authentication tokens, while the repeater tool enables manual editing and resending of requests to test for vulnerabilities. This process highlights critical web security concepts like the importance of input validation, proper session handling, and HTTPS encryption.

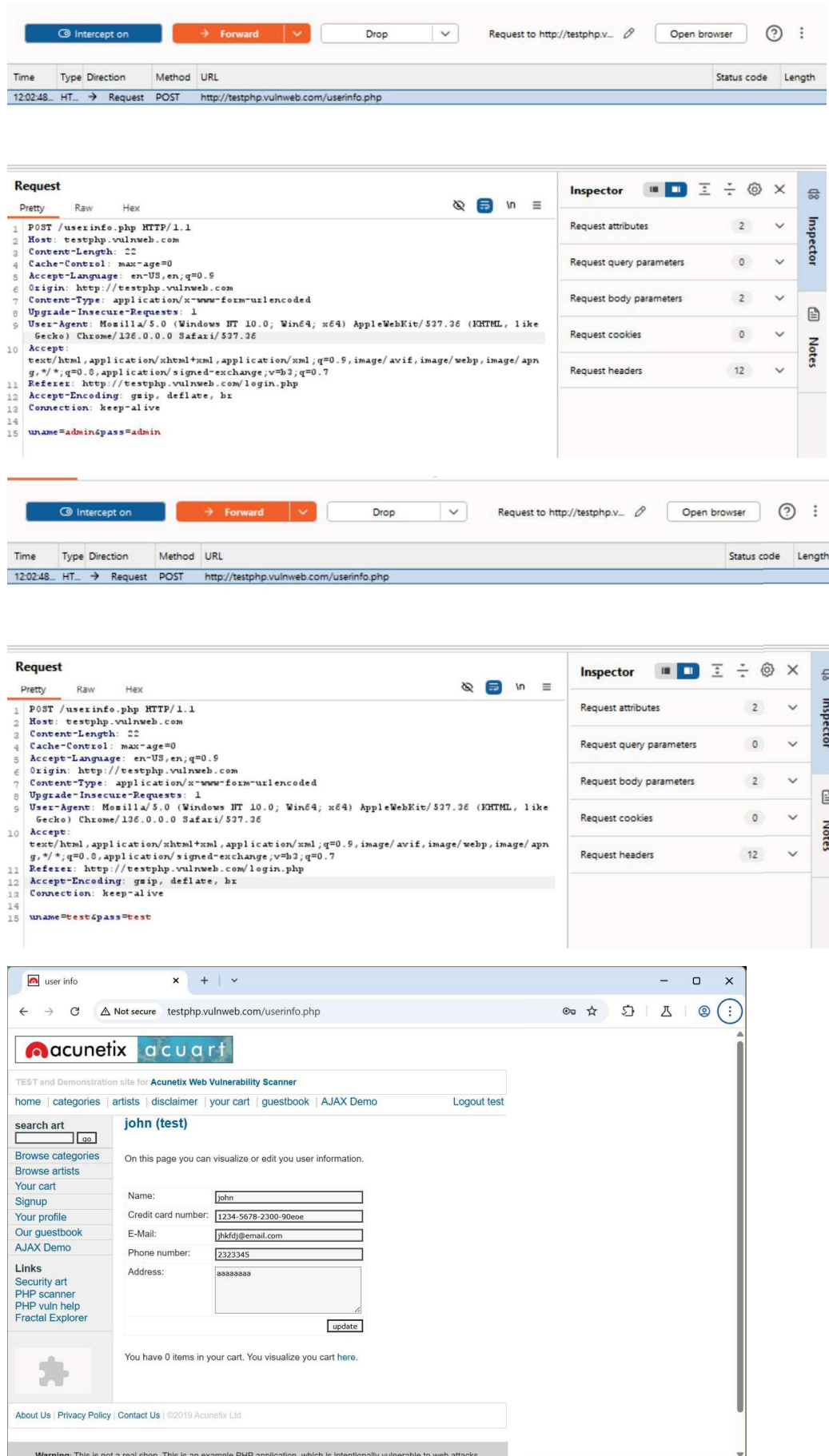
Limitations:

- Requires proper proxy configuration
- May break website functionality if misused
- Cannot intercept properly implemented HTTPS without certificate installation
- Limited to manual testing without premium features

Procedure:

1. Open Burp suite community edition, go to the “Proxy” tab and open a browser.
2. Visit <http://testphp.vulnweb.com> and navigate to the “sign up” page
3. Turn on intercept in burp suite and try to log in with the credentials “admin” for both the username and password
4. Modify the intercepted request in burpsuite and change the login details from “admin” to “test”
5. Forward the request and log in

Output:



Intercept on Forward Drop Request to http://testphp.vulnweb.com/userinfo.php Open browser ?

Time	Type	Direction	Method	URL	Status code	Length
12:02:48...	HT...	→	Request	POST http://testphp.vulnweb.com/userinfo.php		

Request

Pretty Raw Hex

```
1 POST /userinfo.php HTTP/1.1
2 Host: testphp.vulnweb.com
3 Content-Length: 22
4 Cache-Control: max-age=0
5 Accept-Language: en-US,en;q=0.9
6 Origin: http://testphp.vulnweb.com
7 Content-Type: application/x-www-form-urlencoded
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
10 Gecko) Chrome/136.0.0.0 Safari/537.36
11 Accept:
12 text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apn
13 g,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
14 Referer: http://testphp.vulnweb.com/login.php
15 Accept-Encoding: gzip, deflate, br
16 Connection: keep-alive
17
18 username=admin&pass=admin
```

Inspector

Request attributes 2

Request query parameters 0

Request body parameters 2

Request cookies 0

Request headers 12

Notes

Intercept on Forward Drop Request to http://testphp.vulnweb.com/userinfo.php Open browser ?

Time	Type	Direction	Method	URL	Status code	Length
12:02:48...	HT...	→	Request	POST http://testphp.vulnweb.com/userinfo.php		

Request

Pretty Raw Hex

```
1 POST /userinfo.php HTTP/1.1
2 Host: testphp.vulnweb.com
3 Content-Length: 22
4 Cache-Control: max-age=0
5 Accept-Language: en-US,en;q=0.9
6 Origin: http://testphp.vulnweb.com
7 Content-Type: application/x-www-form-urlencoded
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
10 Gecko) Chrome/136.0.0.0 Safari/537.36
11 Accept:
12 text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apn
13 g,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
14 Referer: http://testphp.vulnweb.com/login.php
15 Accept-Encoding: gzip, deflate, br
16 Connection: keep-alive
17
18 username=test&pass=test
```

Inspector

Request attributes 2

Request query parameters 0

Request body parameters 2

Request cookies 0

Request headers 12

Notes

user info

Not secure testphp.vulnweb.com/userinfo.php

Acunetix acuart

TEST and Demonstration site for Acunetix Web Vulnerability Scanner

home categories artists disclaimer your cart guestbook AJAX Demo Logout test

search art

john (test)

On this page you can visualize or edit you user information.

Name: john

Credit card number: 1234-5678-2300-9000

E-Mail: jhkdj@email.com

Phone number: 2323345

Address: aaaaaaaa

update

You have 0 items in your cart. You visualize you cart here.

About Us | Privacy Policy | Contact Us | ©2019 Acunetix Ltd

Warning: This is not a real shop. This is an example PHP application, which is intentionally vulnerable to web attacks.

Lab 5

Using Metasploit to exploit vulnerabilities

Aim:

To use metasploit's automated vulnerability exploitation system

Theory:

Metasploit provides a robust platform for simulating real-world attacks and testing defenses. As the most widely used penetration testing framework, Metasploit offers exploit modules, payloads, and auxiliary tools that enable security professionals to safely assess system vulnerabilities. For email infrastructure testing, Metasploit includes specialized SMTP modules that can verify open relays, enumerate users (via VRFY/EXPN commands), and test for vulnerabilities like buffer overflows in mail servers. The framework's modular design allows testers to chain exploits, pivot through networks, and demonstrate potential attack paths while maintaining full control over the testing process.

Limitations:

- Requires proper authorization for ethical use
- SMTP modules mainly target legacy vulnerabilities
- False positives possible in user enumeration
- Limited against modern, well-secured mail servers
- Can disrupt services if used aggressively

Procedure:

1. Start Kali Linux and metasploitable 2 virtual machines with network adapter on "bridged mode"
2. Start msfconsole on kali and find the smtp user enumeration exploit using the search command
3. Select the user enumeration command and set RHOSTS to be the IP address of the Metasploitable2 Virtual Machine
4. Launch the exploit and wait for it to complete

Output:

```
msf6 > grep scanner search smtp
4 auxiliary/scanner/http/gavazzi_em_login_loot . normal No Carlo Gavazzi Energy Meters - Login Brute Force, Extract Info and
Dump Plant Database
37 auxiliary/scanner/smtp/smtp_version . normal No SMTP Banner Grabber
38 auxiliary/scanner/smtp/smtp_ntlm_domain . normal No SMTP NTLM Domain Extraction
39 auxiliary/scanner/smtp/smtp_relay . normal No SMTP Open Relay Detection
41 auxiliary/scanner/smtp/smtp_enum . normal No SMTP User Enumeration Utility
66 auxiliary/scanner/http/wp_easy_wp_smtp 2020-12-06 normal No WordPress Easy WP SMTP Password Reset
msf6 > use 41
```

```
msf6 auxiliary(scanner/smtp/smtp_enum) > set rhosts 192.168.1.106
rhosts => 192.168.1.106
msf6 auxiliary(scanner/smtp/smtp_enum) > exploit
[*] 192.168.1.106:25 - 192.168.1.106:25 Banner: 220 metasploitable.localdomain ESMTP Postfix (Ubuntu)
[+] 192.168.1.106:25 - 192.168.1.106:25 Users found: , backup, bin, daemon, distccd, ftp, games, gnats, irc, libuuid, list, lp, mail, man, mysql, news, nobod
y, postfix, postgres, postmaster, proxy, service, sshd, sync, sys, syslog, user, uucp, www-data
[*] 192.168.1.106:25 - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf6 auxiliary(scanner/smtp/smtp_enum) > 
```

Lab 6

SQLi on a vulnerable website

Aim:

To execute SQLi on an vulnerable web application

Theory:

Metasploitable 2 and DVWA (Damn Vulnerable Web Application) serve as controlled test environments for studying SQL injection vulnerabilities. These platforms contain deliberately implemented security flaws that allow for safe experimentation with database exploitation techniques without affecting production systems.

Metasploitable 2 includes multiple vulnerable web applications with various SQL injection points, while DVWA provides configurable security levels to demonstrate different vulnerability scenarios. Both environments enable researchers to observe how unsanitized user input can lead to unauthorized database access, data leakage, and authentication bypass.

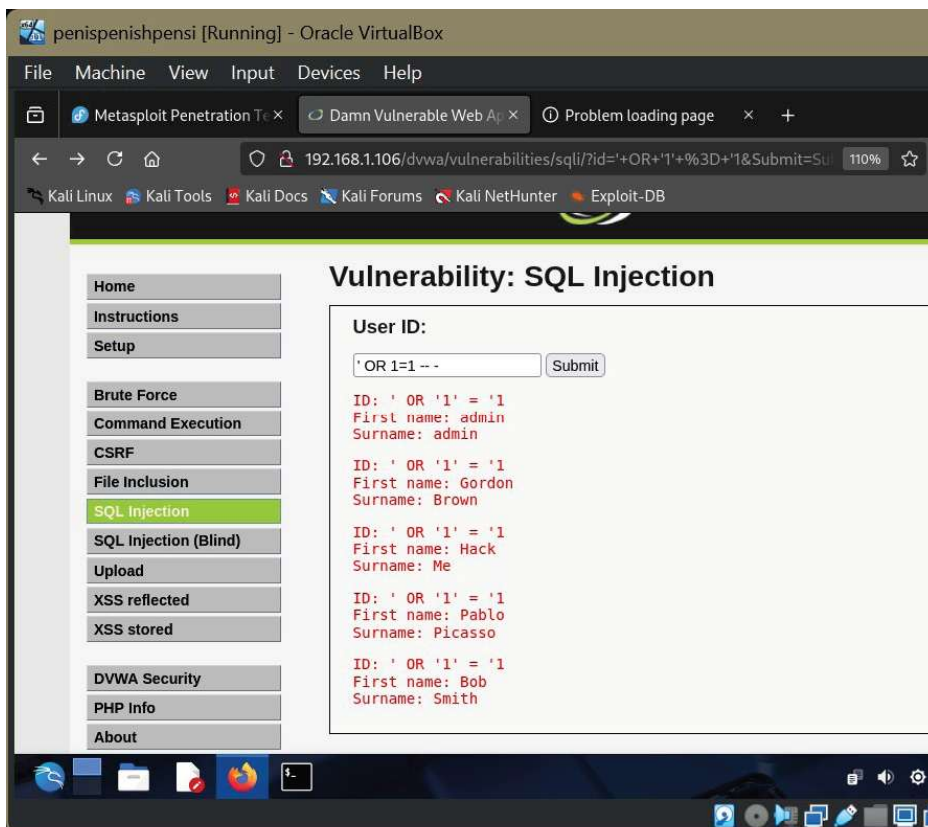
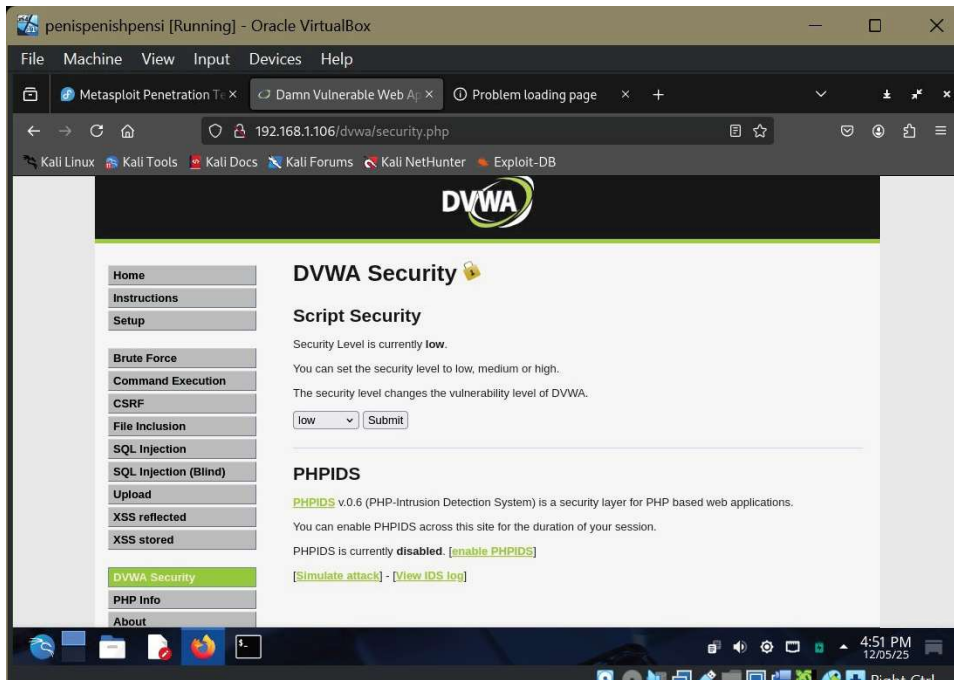
Key Characteristics of the Test Environments

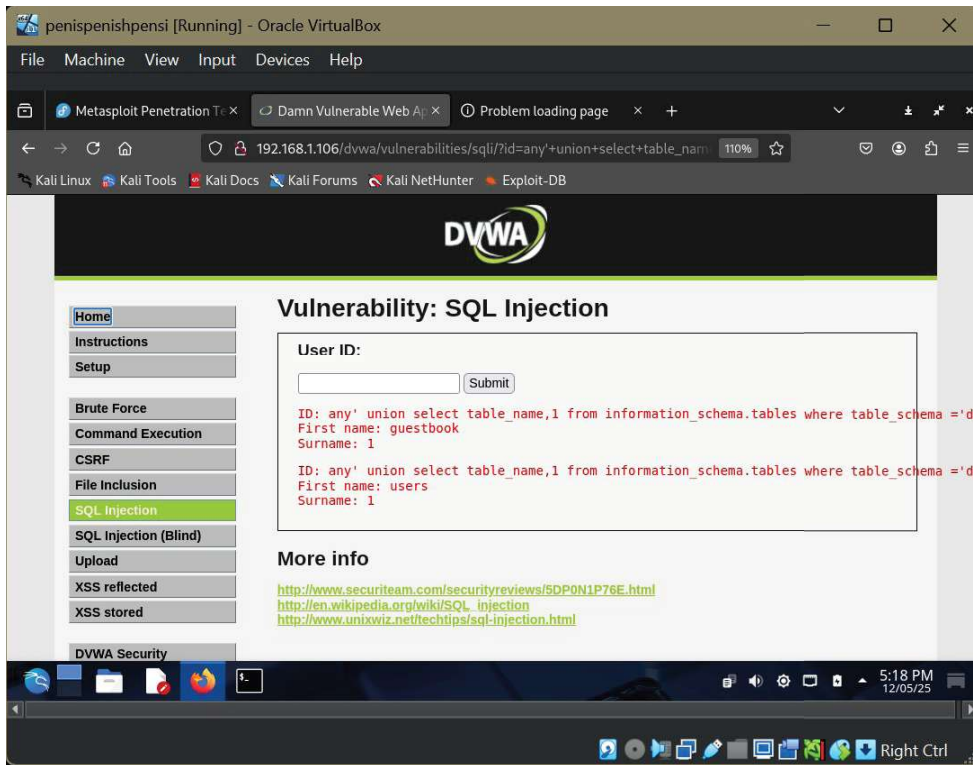
- Pre-configured SQL injection vulnerabilities
- Multiple difficulty levels for progressive learning
- Real-world database structures and error responses
- Isolation from production networks
- Support for both manual and automated testing approaches

Procedure:

1. Start Kali Linux and metasploitable 2 virtual machines with network adapter on “bridged mode”
2. On Kali, connect to the Metasploitable 2 Virtual machine using it’s IP (found by using ifconfig)
3. Navigate to the DVWA section, log in with the credentials “admin” & “password”.
4. Set the DVWA security level to “Low” and navigate to “SQL injection”.
5. Enter Malicious Payload “ ‘ OR 1=1 -- - ” to check for SQLi vulnerabilities.
6. Use the payload “any' union select table_name,1 from information_schema.tables where table_schema='dvwa'#” to find information of the database schema

Output





Lab 7

Executing XSS on a vulnerable website

Aim:

To demonstrate the existence of XSS (Cross-Site scripting) vulnerabilities on a website

Theory:

The Metasploitable vulnerable testing environment contains multiple web applications configured with intentional reflected XSS vulnerabilities. These flaws occur when user-supplied input is directly included in server responses without proper output encoding or input validation. The reflected XSS vulnerabilities in this environment demonstrate how malicious scripts can be delivered through crafted URLs or form submissions, with the payload executing immediately in the victim's browser when the response is rendered.

Testing Environment Characteristics:

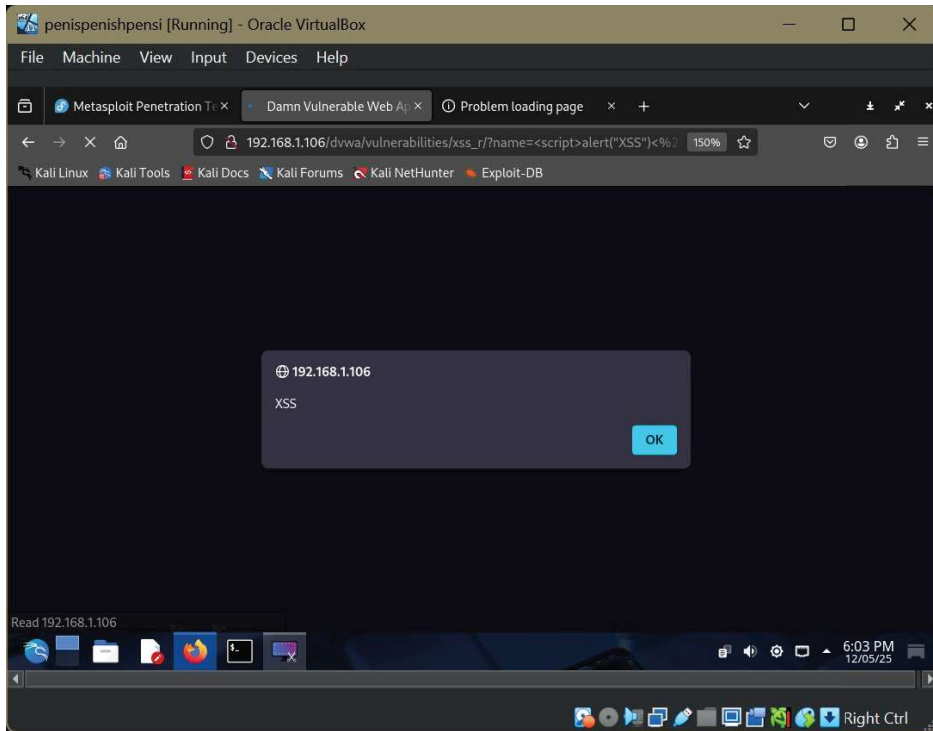
- Web applications specifically configured with reflected XSS vulnerabilities
- Multiple injection points including search fields and URL parameters
- Configurable security levels demonstrating different vulnerability scenarios
- Safe isolation from production systems and networks
- Clear visualization of attack vectors and execution contexts

This controlled setup allows for practical examination of reflected XSS attack methodologies while maintaining a secure testing environment. The vulnerable applications provide observable examples of how improper input handling leads to client-side script execution, enabling researchers to study both the vulnerabilities and potential mitigation techniques.

Procedure:

1. Start Kali Linux and metasploitable 2 virtual machines with network adapter on "bridged mode"
2. On Kali, connect to the Metasploitable 2 Virtual machine using its IP (found by using ifconfig)
3. Navigate to the DVWA section, log in with the credentials "admin" & "password".
4. Set the DVWA security level to "Low" and navigate to "XSS reflected"
5. Input the malicious prompt "<script>alert('XSS')</script>"

Output:



Lab 8

OS command injection using DVWA

Aim:

To inject commands into the linux server running metasploitable2

Theory:

OS Command Injection Testing in DVWA

The Damn Vulnerable Web Application (DVWA) provides a controlled environment for studying OS command injection vulnerabilities, where malicious actors exploit web applications to execute arbitrary operating system commands on the host server. DVWA's command injection module demonstrates how improper input validation in system functions (e.g., ping, whois) allows attackers to inject and execute unauthorized shell commands.

Key Features for Testing:

- Simulated vulnerable input fields accepting user-supplied commands
- Demonstrates concatenation attacks (e.g., ; cat /etc/passwd)
- Shows impact of command chaining (&&, |, ||)
- Configurable security levels (Low/Medium/High) to test bypass techniques
- Safe, isolated environment for studying server-side execution

Procedure:

1. Start Kali Linux and metasploitable 2 virtual machines with network adapter on “bridged mode”
2. On Kali, connect to the Metasploitable 2 Virtual machine using it’s IP (found by using ifconfig)
3. Navigate to the DVWA section, log in with the credentials “admin” & “password”.
4. Set the DVWA security level to “Low” and navigate to “Command Injection”
5. Get the passwords to all the users by checking the passwd file (cat /etc/passwd)

Output:

